SLAM : Projet Agora Sprint 5, demande 5

Dashboard et requêtes personnalisées avec DQL et QueryBuilder

Auteur : *Yannick Narbey d'après un tutoriel de Marianne Delio*

Pré-requis :

- Symfony : contrôleurs, routes, templates twig
- Agora Sprint 5 demande 4 : création de BDD et génération de formulaire CRUD avec Doctrine

TABLE DES MATIERES

1. Contexte et objectifs ____ *2*

2. Mise en œuvre ____ *3*

2.1 Dashboard généré sur la base d'une requête SQL ____ **3**

2.2 Dashboard généré sur la base d'une requête DQL ____ **7**

2.3 Dashboard généré avec Query Builder ____ **10**

2.4 Dashboard généré sur la base des méthodes standards des repository ____ **12**

2.4.1 Création du contrôleur et de sa vue ____ 12

2.4.2 Modifier le contrôleur pour récupérer les données depuis l'entité Tournoi ____ 12

2.4.3 Amelioration du dashboard pour afficher le nombre de participants ____ 15

2.4.4 Ajout d'un filtre sous forme d'une liste déroulante ____ 16

2.4.5 Ajout d'un composant graphique pour faciliter la lecture du dashboard ____ 18

3. Conclusion ____ *22*

1. Contexte et objectifs

Le directeur de l'association Agora demande de pouvoir disposer d'un tableau de bord (Dashboard) pour suivre les tournoi, le nombre d'inscrit, voir de pouvoir appliquer des filtres sur les dates et les catégories de tournois.

L'ESN Logma est toujours en charge d'ajouter ces fonctionnalités sur l'application Agora.

Dans cette itération, vous êtes chargé de réaliser ces Dashboards à l'aide du composant Doctrine. Plusieurs techniques étant possibles (SQL, DQL, Quesry byuilder), il est décider de les réaliser pour comparer leur efficacité.

Symfony (rappel)

Symfony est un ensemble de composants PHP, un **framework d'application Web**, une philosophie et une communauté - tous travaillant ensemble en harmonie. (site officiel)
Symfony est un framework basé sur le modèle MVC qui permet de réaliser des sites complexes rapidement, mais de façon structurée et avec un code clair et maintenable.
SensioLabs est le créateur du framework PHP Symfony. Le nom SensioLabs vient du mot espagnol "sencillo" qui signifie "simple". Fabien Potencier est le fondateur et chef de projet Symfony.

https://symfony.com/

Doctrine (rappel)

Symfony est livré par défaut avec l'**ORM** Doctrine. Un ORM (pour **Object-Relation Mapper**, « lien objet-relation ») est un programme qui se place en interface entre un programme applicatif et une base de données relationnelle pour simuler une base de données orientée objet. Il définit des **correspondances** entre les **schémas de la base** de données et les **classes** du programme applicatif.

https://www.doctrine-project.org

Entité (rappel)

Les **données** de la base de données sont des **objets**. Un objet dont l'enregistrement est confié à l'ORM se nomme une **entité** (entity en anglais). On dit également **persister une entité**, plutôt qu'enregistrer une entité.

Attributs PHP et Doctrine (rappel)

Doctrine utilise les attributs PHP 8 afin de définir la table correspondant à une entité, définir un identifiant (id) en auto-incrément, nommer les colonnes, etc. Ces informations se nomment les metadata de l'entité.
Ajouter les metadata à un objet, s'appelle **mapper l'objet, c'est**-à-dire faire le lien entre l'objet et la représentation physique qu'utilise Doctrine (une table de la base de données relationnelle).

DQL et Query Builder

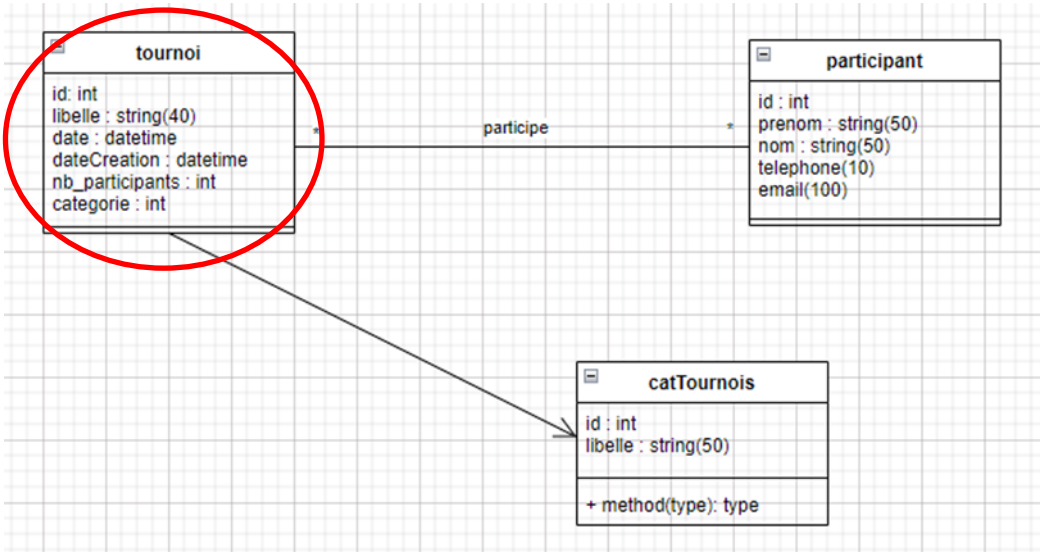
Le DQL (Doctrine Query Language) est un langage de requête orienté objet propre à Doctrine, qui permet d'interroger la base de données en manipulant les entités PHP plutôt que les tables SQL.
Le QueryBuilder, quant à lui, est une interface fluide proposée par Doctrine pour construire dynamiquement des requêtes DQL en PHP, ce qui le rend idéal pour générer des requêtes complexes ou conditionnelles de manière programmatique.

https://www.doctrine-project.org/projects/doctrine-orm/en/3.5/reference/dql-doctrine-query-language.html

Le développement de l'application se fera dans un cadre de travail agile **SCRUM**.

2. Mise en œuvre

Nous disposons du diagramme UML sur lequel a été basé la création des entités et la migration en BDD, c'est sur cette base que seront rédigés les requetés SQL, DQL et QueryBuilder. Pour répondre à la demande de la direction de l'association de disposer d'un Dashboard pour lister les tournois organisés après une date.



2.1 Dashboard généré sur la base d'une requête SQL

Ajoutons la méthode findAllAfterThanDateSQL() dans TournoiRepository.php qui contiendra la requete « SELECT * FROM tournoi t WHERE t.date >= :datemax ORDER BY t.date ASC ».

```
    /**
     * @return Tournoi[]
     */
    public function findAllAfterThanDateSQL($datemax): array
    {
        $conn = $this->getEntityManager()->getConnection();

        $sql = 'SELECT * FROM tournoi t
                WHERE t.date >= :datemax
                ORDER BY t.date ASC';
        $stmt = $conn->prepare($sql);
        $resultat = $stmt->executeQuery(['datemax' => $datemax]);

        return $resultat->fetchAllNumeric();
    }
```

Cette méthode renvoie un tableau de données brutes ($t[1], $t[2], etc.) contenant de l'information sur les tournois dont les dates sont postérieures à $datemax passée en paramètre.

Créer un contrôleur src/controller/TournoisSQLStatController.php qui va utiliser cette méthode findAllAfterThanDateSQL() et appeler la vue tournoi_stat/sql.html.twig

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;
```

```php
use Symfony\Component\Routing\Annotation\Route;
use App\Repository\TournoiRepository;

class TournoiSQLStatController extends AbstractController
{
    #[Route('/statSql', name: 'app_stat_sql_tournoi')]
    public function indexSQL(Request $request, TournoiRepository $tournoiRepository):
Response
    {
        // On récupère une date passée en GET ou on met une valeur par défaut
        $datemax = $request->query->get('datemax', '2025-01-01');

        // Appel à la méthode SQL du repository
        $tournois = $tournoiRepository->findAllAfterThanDateSQL($datemax);

        // Transformation du résultat SQL brut en tableau exploitable
        $stats = [];
        foreach ($tournois as $t) {
            $stats[] = [
                'libelle' => $t[1] ?? '(non défini)', // dépend de l'ordre des colonnes
                'categorie' => $t[2] ?? '(catégorie inconnue)',
                'date' => $t[3] ?? '(pas de date)',
            ];
        }

        return $this->render('tournoi_stat/sql.html.twig', [
            'stats' => $stats,
            'datemax' => $datemax,
            'mode' => 'SQL',
        ]);
    }
}
```

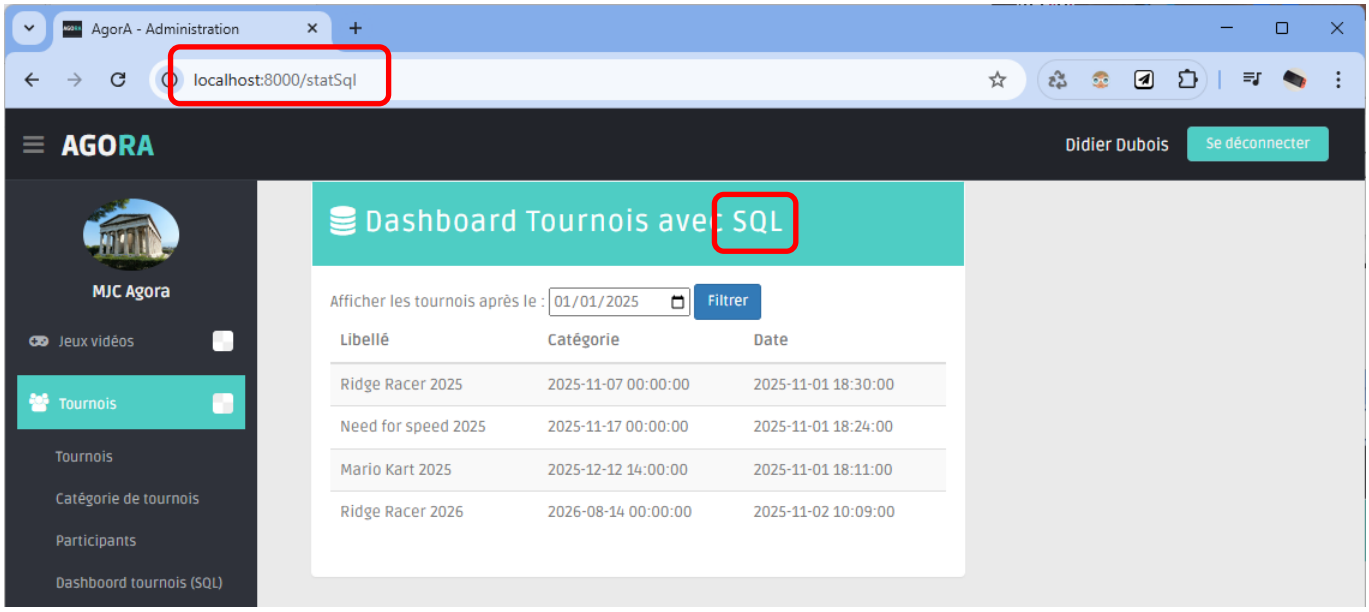
Enfin, créer la vue tournoi_stat/sql.html.twig

```twig
{% extends 'base.html.twig' %}

{% block title %}Statistiques {{ mode }}{% endblock %}

{% block body %}
<div class="col-sm-8">
    <section class="panel">
        <div class="chat-room-head">
            <h3>
                <i class="fa fa-database"></i> Dashboard Tournois avec {{ mode }}
            </h3>
        </div>

        <div class="panel-body">
            <form method="get" action="{{ app.request.pathinfo }}" class="mb-3">
                <label for="datemax">Afficher les tournois après le :</label>
                <input type="date" name="datemax" id="datemax" value="{{ datemax }}" />
                <button type="submit" class="btn btn-primary btn-sm">Filtrer</button>
```

La variable mode permettra de préciser si le dashboard est produit avec SQL, DQL ou QB

```
            </form>

            <table class="table table-striped table-advance table-hover">
                <thead>
                    <tr>
                        <th>Libellé</th>
                        <th>Catégorie</th>
                        <th>Date</th>
                    </tr>
                </thead>
                <tbody>
                    {% for t in stats %}
                        <tr>
                            <td>{{ t.libelle }}</td>
                            <td>{{ t.categorie }}</td>
                            <td>{{ t.date }}</td>
                        </tr>
                    {% else %}
                        <tr>
                            <td colspan="3"><em>Aucun tournoi trouvé après {{ datemax }}</em></td>
                        </tr>
                    {% endfor %}
                </tbody>
            </table>
        </div>
    </section>
</div>
{% endblock %}
```

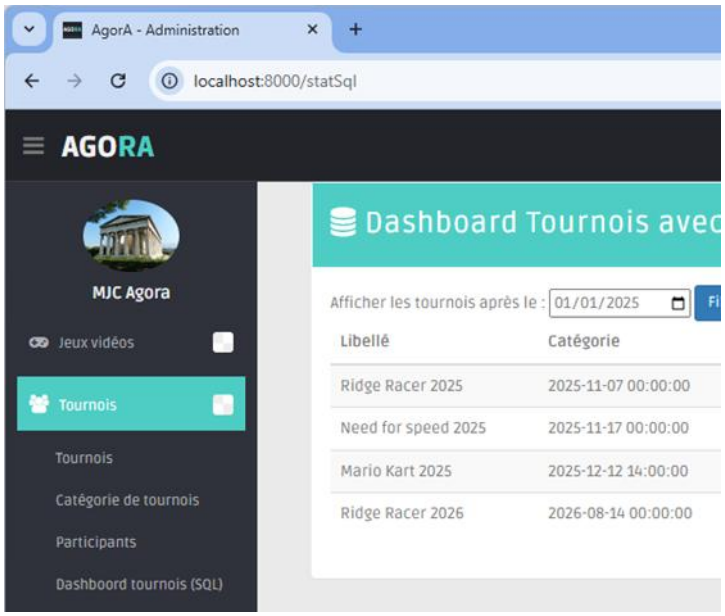
Tester avec http://localhost:8000/statSql



Modifier le menu (templates/menu.html/twig) poru ajouter une nouvelle entrée qui lance ce dashboard :

```
        <li class="sub-menu">
            <a{% if menuActif is defined and menuActif == 'Tournois' %} class="active" {% endif %} href="javascript:;">
                <i class="fa fa-group"></i>
                <span>Tournois</span>
```

```
            </a>
            <ul class="sub">
                <li><a href="{{path('app_tournoi_index')}}">Tournois</a></li>
                <li><a href="{{path('app_cat_tournois_index')}}">Catégorie de tournois</a></li>
                <li><a href="{{path('app_participant_index')}}">Participants</a></li>
                <li><a href="{{path('app_stat_sql_tournoi')}}">Dashboard tournois (SQL)</a></li>
            </ul>
        </li>
```



Pour finir, il faut veiller à maintenir le menu Tournois ouvert quand on demande l'ouverture de ce nouvel écran :

TournoiSQLStatController.php | services.yaml | menu.html.twig | index.html.twig | TournoiStatController.pl

C: > wamp64 > www > agoram55 > src > Controller > TournoiSQLStatController.php

```php
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;
use App\Repository\TournoiRepository;

class TournoiSQLStatController extends AbstractController
{
    #[Route('/statSql', name: 'app_stat_sql_tournoi')]
    public function indexSQL(Request $request, TournoiRepository $tournoiRepository): Respons
    {
        // On récupère une date passée en GET ou on met une valeur par défaut
        $datemax = $request->query->get('datemax', '2025-01-01');

        // Appel à la méthode SQL du repository
        $tournois = $tournoiRepository->findAllAfterThanDateSQL($datemax);

        // Transformation du résultat SQL brut en tableau exploitable
        $stats = [];
        foreach ($tournois as $t) {
            $stats[] = [
                'libelle' => $t[1] ?? '(non défini)', // dépend de l'ordre des colonnes
                'categorie' => $t[2] ?? '(catégorie inconnue)',
                'date' => $t[3] ?? '(pas de date)',
            ];
        }

        return $this->render('tournoi_stat/sql.html.twig', [
            'stats' => $stats,
            'datemax' => $datemax,
            'menuActif' => 'Tournois',
        ]);
    }
}
```

Li 35, Col 11 (28 sélectionné) Espaces : 4 UTF-8 LF {} PHP

2.2 Dashboard généré sur la base d'une requête DQL

Ajoutons la méthode findAllAfterThanDateDQL() dans TournoiRepository.php . La requête DQL est proche du SQL à la différence majeure prêt qu'elle s'appuie sur l'entité app\entity\Tournoi à la place de s'appuyer sur la table tournoi.

```php
    /**
    * @return Tournoi[]
    */
    public function findAllAfterThanDateDQL($datemax): array
    {
        $entityManager = $this->getEntityManager();

        // ce n'est pas du SQL mais du DQL : Doctrine Query Language
        // il s'agit en fait d'une requête classique mais qui référence l'objet au lieu
de la table
        $query = $entityManager->createQuery(
            'SELECT t
            FROM App\Entity\Tournoi t
```

```
            WHERE t.date >= :datemax
            ORDER BY t.date ASC'
        )->setParameter('datemax', $datemax);

        // retourne un tableau d'objets de type Tournoi
        return $query->getResult();
    }
```

cette requête DQL manipule par défaut l'entité tournoi puisqu'on est dans son repository et a pour but de sélectionner tous les objets tournoi dont la propriété date est supérieure ou égale à la valeur :datemax, et les trie par date croissante

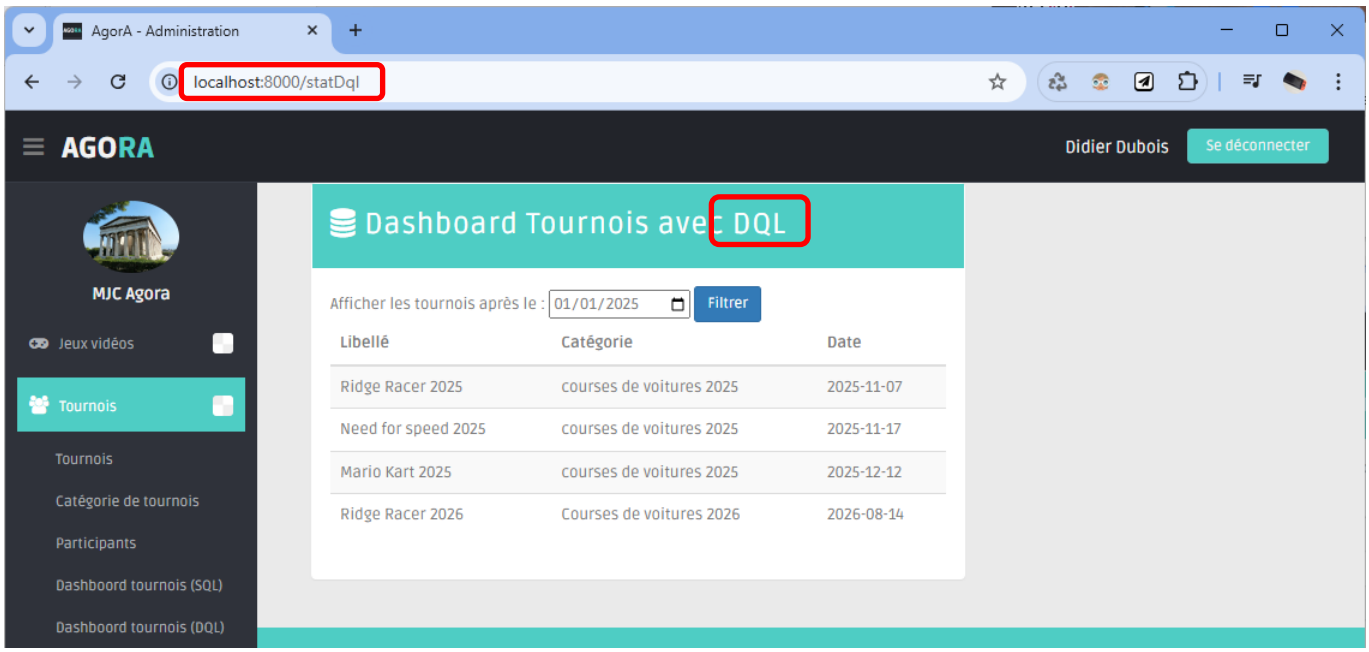
Cette méthode renvoie une collection d'objets dont la date est supérieure à $datemax passé en paramètre.

Créer un contrôleur src/controller/TournoisDQLStatController.php qui va utiliser cette méthode findAllAfterThanDateDQL() et appeler la vue tournoi_stat/sql.html.twig (la même vue que pour le SQL).

```php
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;
use App\Repository\TournoiRepository;

class TournoiDQLStatController extends AbstractController
{
    #[Route('/statDql', name: 'app_stat_dql_tournoi')]
    public function indexDQL(Request $request, TournoiRepository $tournoiRepository): Response
    {
        // On récupère une date passée en GET ou on met une valeur par défaut
        $datemax = $request->query->get('datemax', '2025-01-01');

        // Appel à la méthode DQL du repository
        $tournois = $tournoiRepository->findAllAfterThanDateDQL($datemax);

        // Transformation du résultat (objets Tournoi) en tableau exploitable pour la vue
        $stats = [];
        foreach ($tournois as $t) {
            $stats[] = [
                'libelle' => $t->getLibelle(),
                'categorie' => $t->getCategorie() ? $t->getCategorie()->getLibelle() : '(catégorie inconnue)',
                'date' => $t->getDate() ? $t->getDate()->format('Y-m-d') : '(pas de date)',
            ];
        }

        return $this->render('tournoi_stat/sql.html.twig', [
            'stats' => $stats,
            'datemax' => $datemax,
            'mode' => 'DQL',
            'menuActif' => 'Tournois',
        ]);
    }
}
```

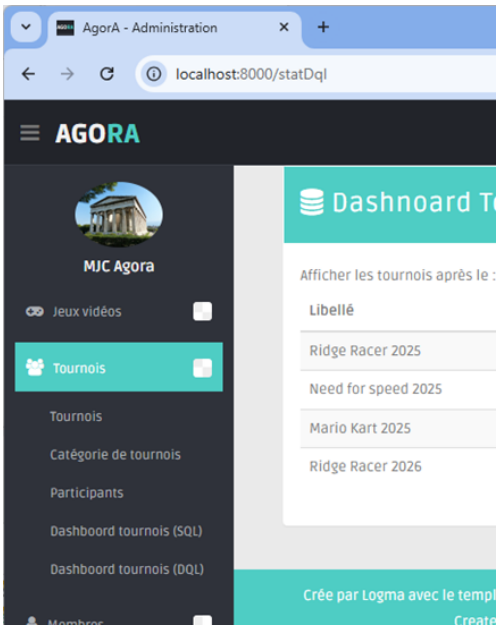
Ce contrôleur réutilise la vue tournoi_stat/sql.html.twig précédemment créée, donc inutile de la recréer.

Tester avec localhost:8000/statDql



Modifier le menu (templates/menu.html/twig),

```
<li class="sub-menu">
    <a{% if menuActif is defined and menuActif == 'Tournois' %} class="active" {% endif %} href="javascript:;">
        <i class="fa fa-group"></i>
        <span>Tournois</span>
    </a>
    <ul class="sub">
        <li><a href="{{path('app_tournoi_index')}}">Tournois</a></li>
        <li><a href="{{path('app_cat_tournois_index')}}">Catégorie de tournois</a></li>
        <li><a href="{{path('app_participant_index')}}">Participants</a></li>
        <li><a href="{{path('app_stat_sql_tournoi')}}">Dashboard tournois (SQL)</a></li>
        <li><a href="{{path('app_stat_dql_tournoi')}}">Dashboard tournois (DQL)</a></li>
    </ul>
</li>
```



2.3 Dashboard généré avec Query Builder

Ajoutons la méthode findAllAfterThanDateQB() dans TournoiRepository.php. a la différence de DQL, une requete QueryBuilder est composé par ajout successifs ->where() puis ->setParameter() puis ->orderBy() …

```
    /**
     * @return Tournoi[]
     */
    public function findAllAfterThanDateQB($datemax): array
    {
        // ce n'est pas du SQL mais QueryBuilder
         $qb = $this->createQueryBuilder('t')
            ->where('t.date >= :datemax')
            ->setParameter('datemax', $datemax)
            ->orderBy('t.date', 'ASC');
        $query = $qb->getQuery();

        // retourne un tableau d'objets de type Tournoi
        return $query->getResult();
    }
```

cette requête QueryBuilder manipule également l'entité tournoi et a pour but de sélectionner tous les tournois dont la propriété date est >= à :datemax et trie par date croissante

Cette méthode renvoie une collection d'objets tournoi dont les dates sont postérieures à $datemax passée en paramètre.

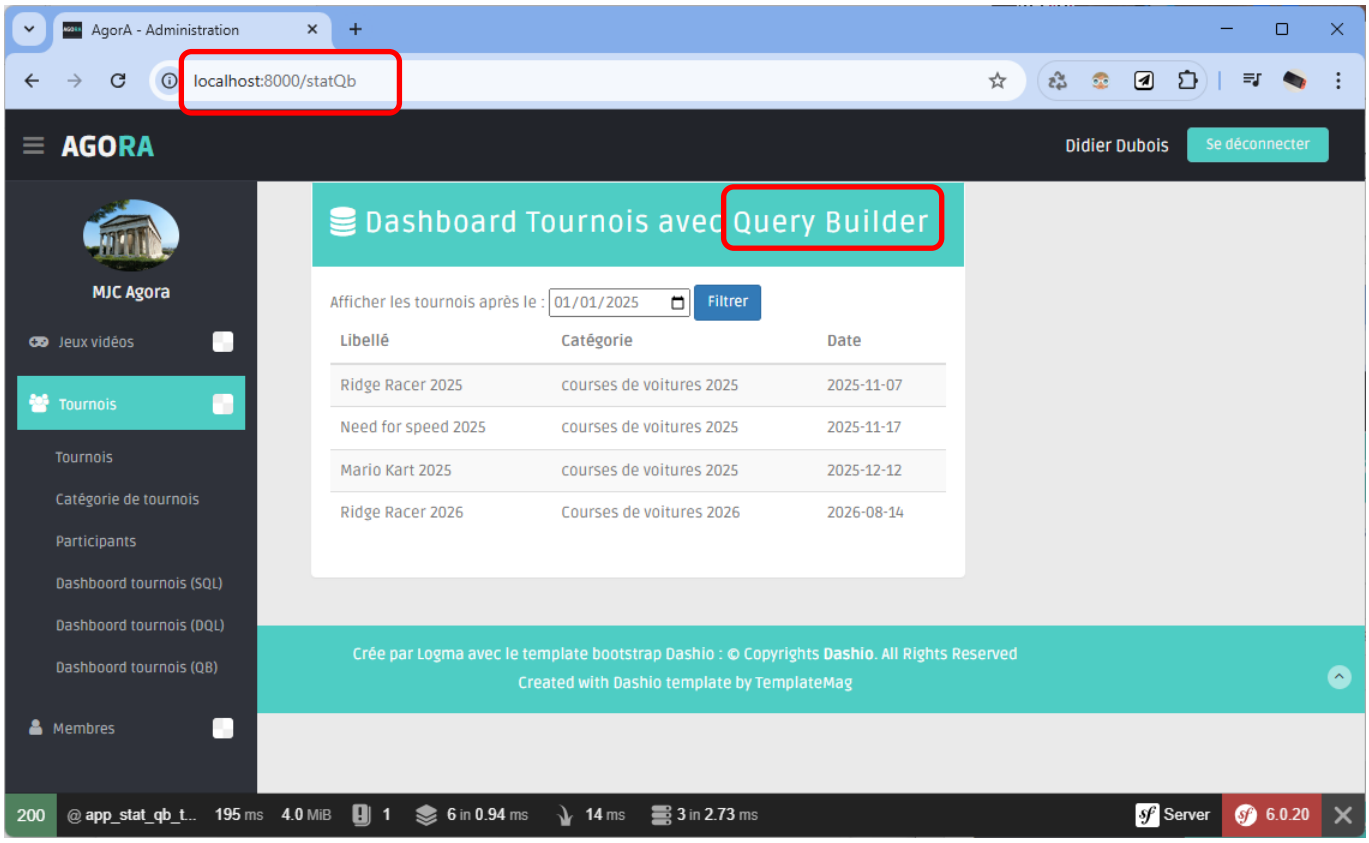
Créer un contrôleur src/controller/TournoisQBStatController.php qui va utiliser cette méthode findAllAfterThanDateQB() et appeler la vue tournoi_stat/sql.html.twig

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;
use App\Repository\TournoiRepository;

class TournoiQBStatController extends AbstractController
{
    #[Route('/statQb', name: 'app_stat_qb_tournoi')]
    public function indexDQL(Request $request, TournoiRepository $tournoiRepository):
Response
    {
        // On récupère une date passée en GET ou on met une valeur par défaut
        $datemax = $request->query->get('datemax', '2025-01-01');

        // Appel à la méthode DQL du repository
        $tournois = $tournoiRepository->findAllAfterThanDateQB($datemax);

        // Transformation du résultat (objets Tournoi) en tableau exploitable pour la
vue
        $stats = [];
        foreach ($tournois as $t) {
```

```
            $stats[] = [
                'libelle' => $t->getLibelle(),
                'categorie' => $t->getCategorie() ? $t->getCategorie()->getLibelle() :
'(catégorie inconnue)',
                'date' => $t->getDate() ? $t->getDate()->format('Y-m-d') : '(pas de
date)',
            ];
        }

        return $this->render('tournoi_stat/sql.html.twig', [
            'stats' => $stats,
            'datemax' => $datemax,
            'mode' => 'Query Builder',
            'menuActif' => 'Tournois',
        ]);
    }
}
```

Le contrôleur réutilise la vue tournoi_stat/sql.html.twig, donc inutile de créer une vue.

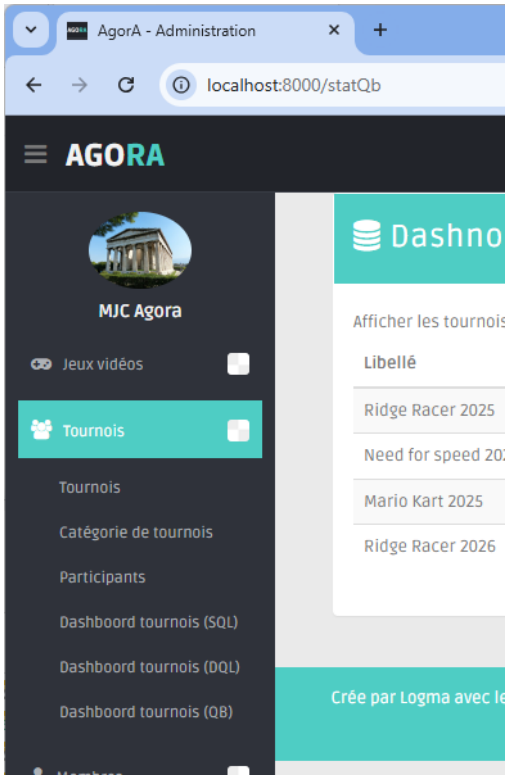
Tester avec http://localhost:8000/statQb



Modifier le menu (templates/menu.html/twig)

```
        <li class="sub-menu">
            <a{% if menuActif is defined and menuActif == 'Tournois' %} class="active" {% endif %} href="javascript:;">
                <i class="fa fa-group"></i>
                <span>Tournois</span>
            </a>
            <ul class="sub">
                <li><a href="{{path('app_tournoi_index')}}">Tournois</a></li>
```

```
            <li><a href="{{path('app_cat_tournois_index')}}">Catégorie de tournois</a></li>
            <li><a href="{{path('app_participant_index')}}">Participants</a></li>
            <li><a href="{{path('app_stat_sql_tournoi')}}">Dashboord tournois</a></li>
            <li><a href="{{path('app_stat_qb_tournoi')}}">Dashboord tournois (QB)</a></li>
        </ul>
    </li>
```



2.4 Dashboard généré sur la base des méthodes standards des repository

Dans cette partie nous répondrons au besoin de la direction de l'association disposer d'un dashboard affichant une synthèse du nombre de participants dans les tournois d'une catégorie
Pour cela, nous utiliserons la méthode findAll(), une des méthodes standard livrées automatiquement lors de la génération du repository de tournoi par Doctrine.

2.4.1 Création du contrôleur et de sa vue

Créons maintenant d'un Controller Symfony:

```
php bin/console make:controller TournoiStatController
```

```
C:\Windows\System32\cmd.e
C:\wamp64\www\agoram55>php bin/console make:controller TournoiStatController

 created: src/Controller/TournoiStatController.php
 created: templates/tournoi_stat/index.html.twig

 Success!

 Next: Open your new controller class and add some pages!

C:\wamp64\www\agoram55>
```

2.4.2 Modifier le contrôleur pour récupérer les données depuis l'entité Tournoi

Modifier src/Controller/TournoiStatController.php et adapte-le ainsi, nous allons utiliser la méthode standard findAll().

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;
use App\Repository\TournoiRepository;

class TournoiStatController extends AbstractController
{
    #[Route('/statTournoi', name: 'app_stat_tournoi_index')]
    public function index(TournoiRepository $tournoiRepository): Response
    {
        // Récupération de tous les tournois
        $tournois = $tournoiRepository->findAll();

        // Construction d'un tableau de données à envoyer à la vue
        $stats = [];
        foreach ($tournois as $tournoi) {
            $stats[] = [
                'libelle' => $tournoi->getLibelle(),
                'categorie' => $tournoi->getCategorie()->getLibelle(),
            ];
        }

        // Envoi des données à la vue
        return $this->render('tournoi_stat/index.html.twig', [
            'stats' => $stats,
        ]);
    }
}
```

Créer la vue tournoi_stat/index.html.twig :

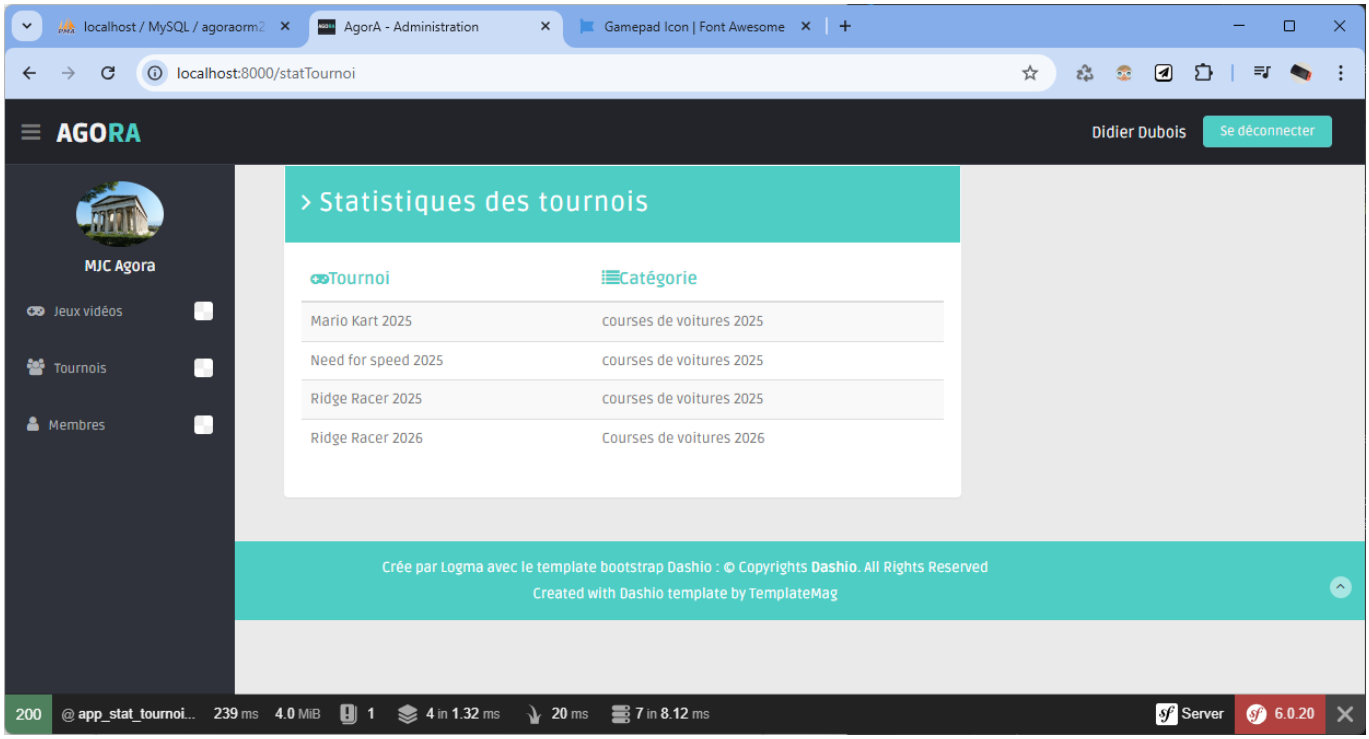
```
{% extends 'base.html.twig' %}

{% block title %}Statistiques des Tournois{% endblock %}

{% block body %}
<h1>Statistiques des Tournois</h1>

<h2>Tableau des tournois</h2>
<table border="1" cellpadding="5" cellspacing="0">
    <tr>
        <th>Tournoi</th>
        <th>Catégorie</th>
    </tr>
    {% for t in stats %}
    <tr>
        <td>{{ t.libelle }}</td>
        <td>{{ t.categorie }}</td>
```

```
    </tr>
    {% endfor %}
</table>

{% endblock %}
```

Charter et franchiser :

```
{% extends 'base.html.twig' %}

{% block body %}
<div class="col-sm-8">
    <section class="panel">
        <div class="chat-room-head">
            <h3><i class="fa fa-angle-right"></i> Statistiques des tournois</h3>
        </div>
        <div class="panel-body">
<table class="table table-striped table-advance table-hover">
    <thead>
        <tr class="tableau-entete">
            <th><i class="fa fa-gamepad"></i>Tournoi</th>
            <th><i class="fa fa-list"></i>Catégorie</th>
        </tr>
    </thead>
    <tbody>
    {% for t in stats %}
    <tr>
        <td>{{ t.libelle }}</td>
        <td>{{ t.categorie }}</td>
    </tr>
    {% endfor %}
</table>
        </div><!-- fin div panel-body-->
    </section><!-- fin section tournois-->
</div><!--fin div col-sm-8-->
{% endblock %}
```

Tester http://localhost:8000/statTournoi



2.4.3 Amelioration du dashboard pour afficher le nombre de participants

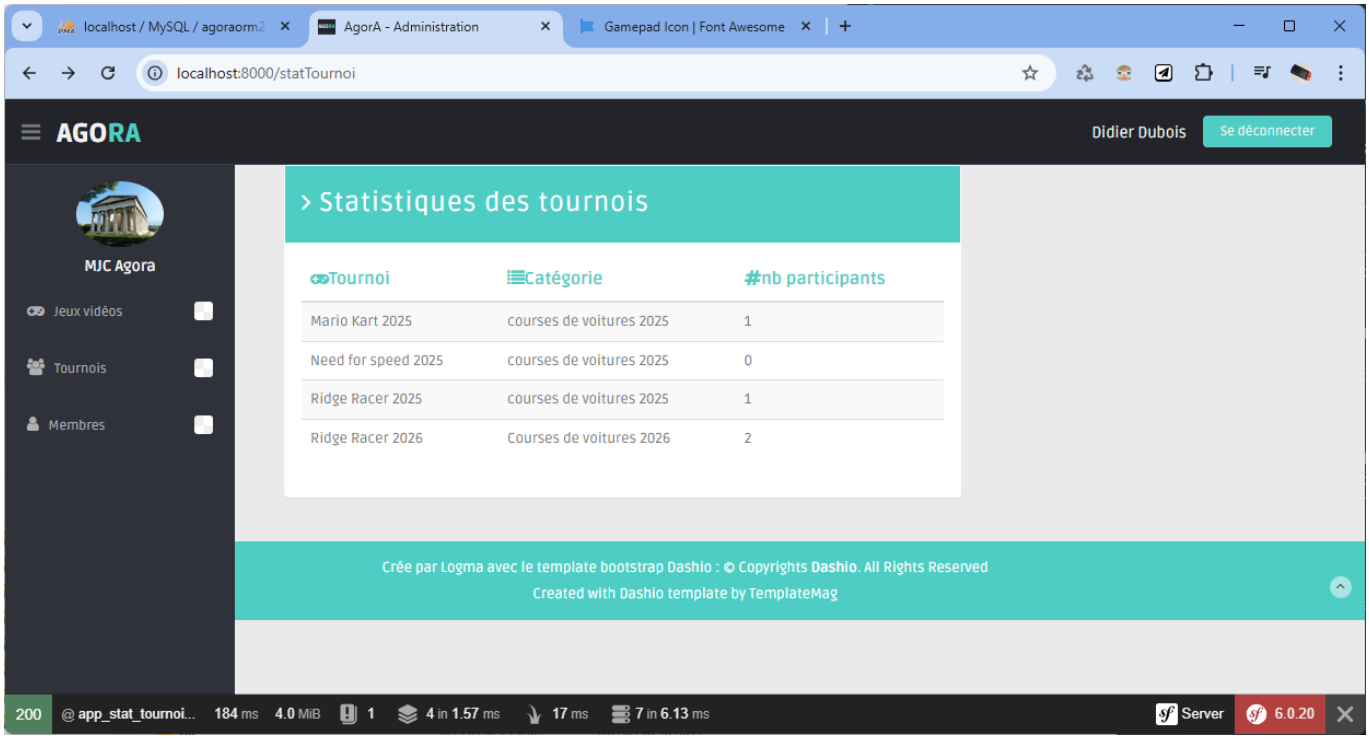
Ajoutons 'nbParticipants' => count($tournoi->getParticipants()), dans le foreach pour ajouter le nb de particpants dans le tableau $stat qui sera envoyé à la vue :

```
        foreach ($tournois as $tournoi) {
            $stats[] = [
                'libelle' => $tournoi->getLibelle(),
                'categorie' => $tournoi->getCategorie()->getLibelle(),
                'nbParticipants' => count($tournoi->getParticipants()),
            ];
        }
```

Et ajouter ces deux lignes dans la vue:

```
<table class="table table-striped table-advance table-hover">
    <thead>
        <tr class="tableau-entete">
            <th><i class="fa fa-gamepad"></i>Tournoi</th>
            <th><i class="fa fa-list"></i>Catégorie</th>
            <th><i class="fa fa-hashtag"></i>nb participants</th>
        </tr>
    </thead>
    <tbody>
    {% for t in stats %}
    <tr>
        <td>{{ t.libelle }}</td>
        <td>{{ t.categorie }}</td>
        <td>{{ t.nbParticipants }}</td>
    </tr>
    {% endfor %}
</table>
```

Tester :



2.4.4 Ajout d'un filtre sous forme d'une liste déroulante

Ajoutons maintenant une liste déroulante au dessus du report pour filtrer sur la catégorie de tournoi.

Il faut donc
- charger l'entité CatTournois,
- filtrer les resultats sur la catégorie choisie
- et ajouter les categories sur chaque ligne du tableau envoyé à la vue :

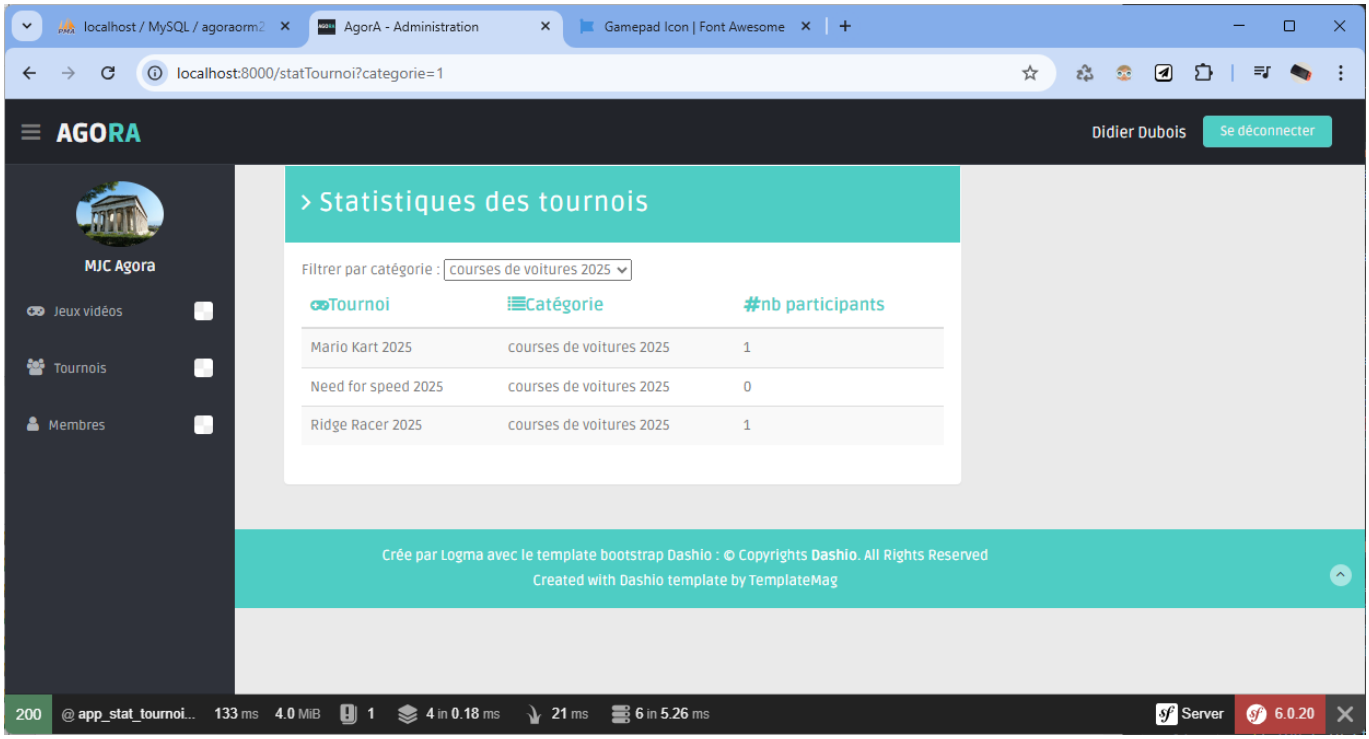
```php
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;
use App\Repository\TournoiRepository;
use App\Repository\CatTournoisRepository;

class TournoiStatController extends AbstractController
{
    #[Route('/statTournoi', name: 'app_stat_tournoi_index')]
    public function index(Request $request, TournoiRepository $tournoiRepository,
CatTournoisRepository $categorieRepository): Response
    {
        $categorieId = $request->query->get('categorie');

        if ($categorieId) {
            $tournois = $tournoiRepository->findBy(['categorie' => $categorieId]);
        } else {
            $tournois = $tournoiRepository->findAll();
        }
```

```
        $stats = [];
        foreach ($tournois as $tournoi) {
            $stats[] = [
                'libelle' => $tournoi->getLibelle(),
                'categorie' => $tournoi->getCategorie()->getLibelle(),
                'nbParticipants' => count($tournoi->getParticipants())
            ];
        }

        $categories = $categorieRepository->findAll();

        return $this->render('tournoi_stat/index.html.twig', [
            'stats' => $stats,
            'categories' => $categories,
            'selectedCategorie' => $categorieId,
        ]);
    }
}
```

Et ajouter le formulaire de filtrage permettant de choisir la catégorie :

```
{% extends 'base.html.twig' %}

{% block body %}
<div class="col-sm-8">
    <section class="panel">
        <div class="chat-room-head">
            <h3><i class="fa fa-angle-right"></i> Statistiques des tournois</h3>
        </div>
        <div class="panel-body">

<form method="get" action="{{ path('app_stat_tournoi_index') }}">
    <label for="categorie">Filtrer par catégorie :</label>
    <select name="categorie" id="categorie" onchange="this.form.submit()">
        <option value="">Toutes</option>
        {% for cat in categories %}
            <option value="{{ cat.id }}" {% if selectedCategorie == cat.id %}selected{% endif %}>
                {{ cat.libelle }}
            </option>
        {% endfor %}
    </select>
</form>

<table class="table table-striped table-advance table-hover">
    <thead>
        <tr class="tableau-entete">
            <th><i class="fa fa-gamepad"></i>Tournoi</th>
            <th><i class="fa fa-list"></i>Catégorie</th>
            <th><i class="fa fa-hashtag"></i>nb participants</th>
        </tr>
    </thead>
    <tbody>
```

Tester :



2.4.5 Ajout d'un composant graphique pour faciliter la lecture du dashboard

Pour finir, ajoutons un élément graphique permettant à l'utilisateur de mieux apprehender le dashboard. Pour cela il suffit d'ajouter le composant graphique "chart.js" dans la vue :

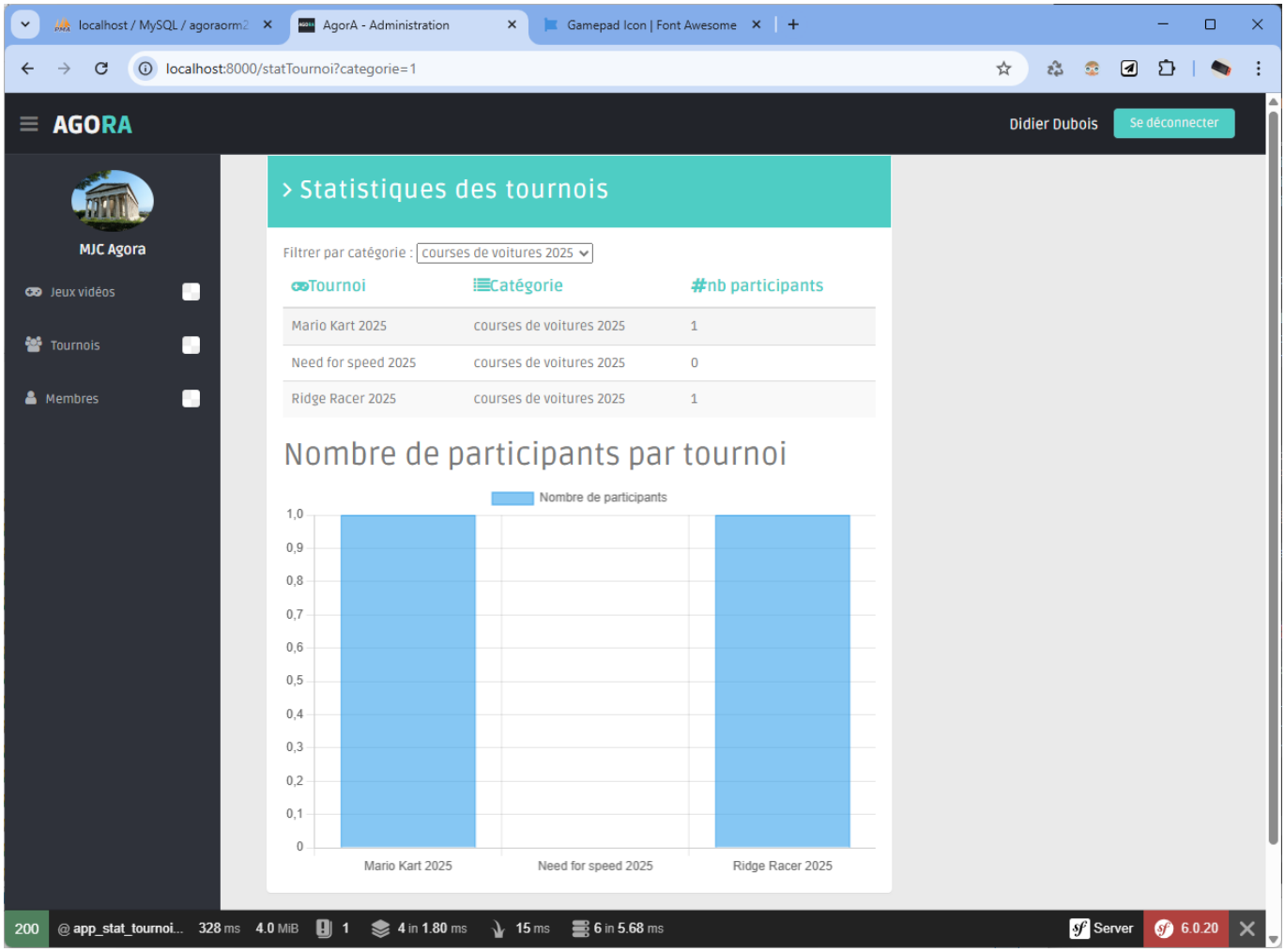
```
    <tbody>
    {% for t in stats %}
    <tr>
        <td>{{ t.libelle }}</td>
        <td>{{ t.categorie }}</td>
        <td>{{ t.nbParticipants }}</td>
    </tr>
    {% endfor %}
</table>

<h2>Nombre de participants par tournoi</h2>
<canvas id="chartParticipants" width="600" height="400"></canvas>

<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
<script>
const ctx = document.getElementById('chartParticipants').getContext('2d');
const chart = new Chart(ctx, {
    type: 'bar',
    data: {
        labels: [{% for t in stats %}'{{ t.libelle }}',{% endfor %}],
        datasets: [{
            label: 'Nombre de participants',
            data: [{% for t in stats %}{{ t.nbParticipants }},{% endfor %}],
            backgroundColor: 'rgba(54, 162, 235, 0.6)',
            borderColor: 'rgba(54, 162, 235, 1)',
            borderWidth: 1
        }]
    },
```

```
    options: {
        scales: {
            y: {
                beginAtZero: true,
                precision: 0
            }
        }
    }
});
</script>
        </div><!-- fin div panel-body-->
    </section><!-- fin section tournois-->
</div><!--fin div col-sm-8-->
{% endblock %}
```

Chart.js est une bibliothèque JavaScript open source qui permet de créer facilement des graphiques interactifs et esthétiques à partir de données HTML ou JSON, directement dans une page web à l'aide de la balise <canvas>.

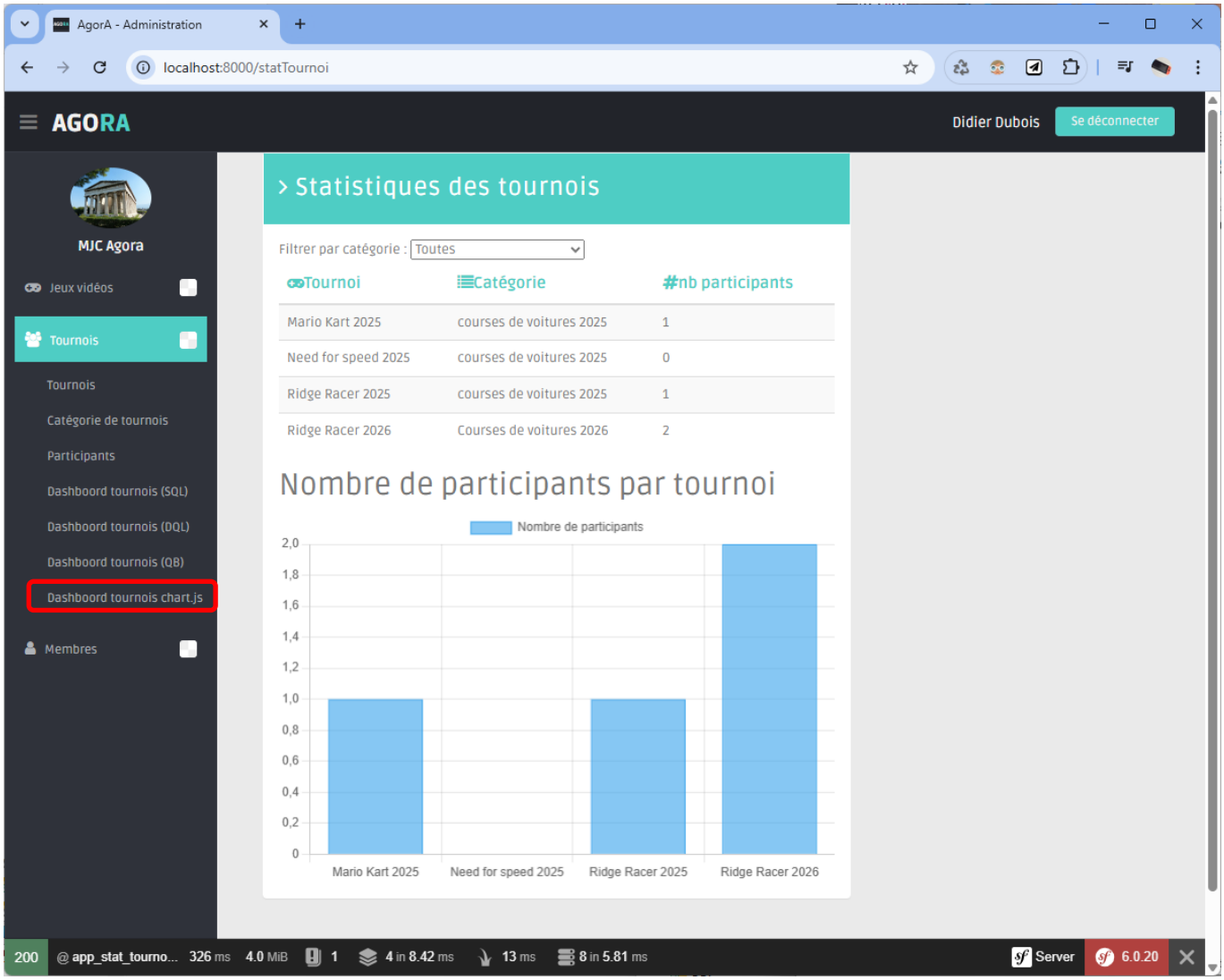
Cette bibliothèque offre une grande variété de graphiques (barres, lignes, secteurs, radar, etc.) et se configure simplement via un objet JavaScript, ce qui en fait un outil idéal pour la visualisation de statistiques dans des applications web comme Symfony.

Tester :



Enfin ajoutons une nouvelle entrée dans le menu :

```
22                <li><a href="{{path('app_marque_index')}}">Marques</a></li>
23                <li><a href="{{path('app_pegi_index')}}">Pegi</a></li>
24            </ul>
25        </li>
26        <li class="sub-menu">
27            <a{% if menuActif is defined and menuActif == 'Tournois' %} class="active" {% end
28                <i class="fa fa-group"></i>
29                <span>Tournois</span>
30            </a>
31            <ul class="sub">
32                <li><a href="{{path('app_tournoi_index')}}">Tournois</a></li>
33                <li><a href="{{path('app_cat_tournois_index')}}">Catégorie de tournois</a></l
34                <li><a href="{{path('app_participant_index')}}">Participants</a></li>
35                <li><a href="{{path('app_stat_sql_tournoi')}}">Dashboord tournois (SQL)</a></
36                <li><a href="{{path('app_stat_dql_tournoi')}}">Dashboord tournois (DQL)</a></
37                <li><a href="{{path('app_stat_qb_tournoi')}}">Dashboord tournois (QB)</a></li
38                <li><a href="{{path('app_stat_tournoi_index')}}">Dashboord tournois chart.js
39            </ul>
40        </li>
41        <li class="sub-menu">
42            <a{% if menuActif is defined and menuActif == 'Membres' %} class="active" {% endi
43                <i class="fa fa-user"></i>
44                <span>Membres</span>
45            </a>
46            <ul class="sub">
```



Veiller à ajuster l'appel de la vue dans le fichier TournoiStatControler.php pour que le menu reste ouvert.

```
        return $this->render('tournoi_stat/index.html.twig', [
            'stats' => $stats,
```

```
            'categories' => $categories,
            'selectedCategorie' => $categorieId,
            'menuActif' => 'Tournois',
        ]);
```

3. Conclusion

Nous avons pu mettre en œuvre un panorama de plusieurs techniques pour réaliser des Dashboards avec Doctrine.

Pour cela, nous avons utilisé les requêtes personnalisées, à mettre en œuvre dans le repository de l'entité concernée, rédigées en :

- SQL
- DQL
- Query Builder

Nous avons également vu comment utiliser la méthode findAll() livrée en standard lors de la génération du repository par Doctrine.

Nous avons vu enfin comment agrémenter un dashboard d'un objet graphique qui permet de faciliter sa compréhension par les utilisateurs.